Name: Abhishek Patil **Class:** CSAIML-A **PRN:** 12414243 **Roll No.:** 08

ASSIGNMENT: 5

1. **Program to print hollow rectangle or square star patterns.**

```
********************
*                  *
*                  *
*                  *
********************
```

Algorithm:

1. Start
2. Input the number of rows and columns
3. Use a loop from i = 1 to rows (outer loop for rows):
   - For each row, use a nested loop from j = 1 to columns (inner loop for columns):
     - If i == 1 or i == rows or j == 1 or j == columns, print *
     - Else, print a space ' '
4. After the inner loop ends, move to the next line (\n)
5. End

Code:

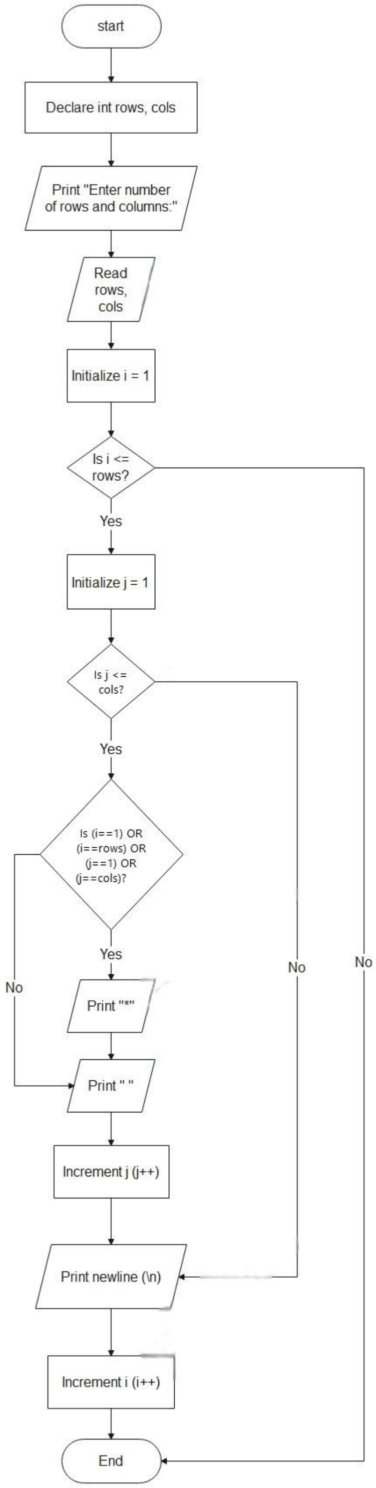
```
#include <stdio.h>

int main() {
   int rows, cols;
   printf("Enter number of rows and columns: ");
   scanf("%d %d", &rows, &cols);

   for(int i = 1; i <= rows; i++) {
      for(int j = 1; j <= cols; j++) {
         if(i == 1 || i == rows || j == 1 || j == cols)
            printf("*");
         else
            printf(" ");
      }
      printf("\n");
```

```
    }
    return 0;
}
```

Flowchart:



2. Write a C program to print pascal triangle up to n rows using loop

Algorithm:

1. Start
2. Input the number of rows for the triangle
3. Use a loop from i = 0 to rows-1 (row by row):
    - Print spaces to center-align the triangle
    - Initialize coef = 1 at the start of each row
    - Use a nested loop from j = 0 to i:
        - Calculate value using formula: coef = coef * (i - j + 1) / j (except for j=0 where coef = 1)
        - Print the coefficient
4. Move to the next line after each row
5. End

Code:

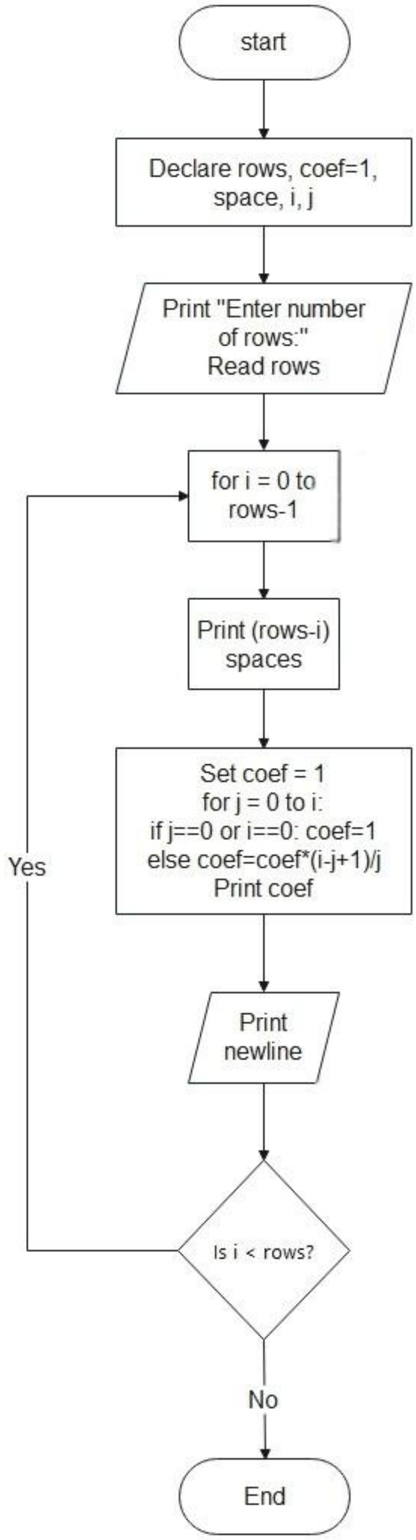
```
#include <stdio.h>

int main() {
    int rows, coef = 1, space, i, j;
    printf("Enter the number of rows: ");
    scanf("%d", &rows);

    for(i = 0; i < rows; i++) {
        for(space = 1; space <= rows - i; space++)
            printf("  ");

        for(j = 0; j <= i; j++) {
            if (j == 0 || i == 0)
                coef = 1;
            else
                coef = coef * (i - j + 1) / j;
            printf("%4d", coef);
        }
        printf("\n");
    }
    return 0;
}
```

Flowchart:


start
Declare rows, coef=1, space, i, j
Print "Enter number of rows:"
Read rows
for i = 0 to rows-1
Print (rows-i) spaces
Set coef = 1
for j = 0 to i:
if j==0 or i==0: coef=1
else coef=coef*(i-j+1)/j
Print coef
Print newline
Is i < rows?
Yes
No
End

3. **Write a c program to display Floyd's Triangle. Floyd's Triangle is a right-angled triangular array of natural numbers. It starts with 1 at the top left and increases sequentially row by row.**

Algorithm:

1. Start
2. Input the number of rows
3. Initialize a variable num = 1
4. Use a loop from i = 1 to rows (for each row):
   - Inside it, run another loop from j = 1 to i:
     - Print the current number num
     - Increment num by 1
5. After the inner loop ends, move to the next line
6. End

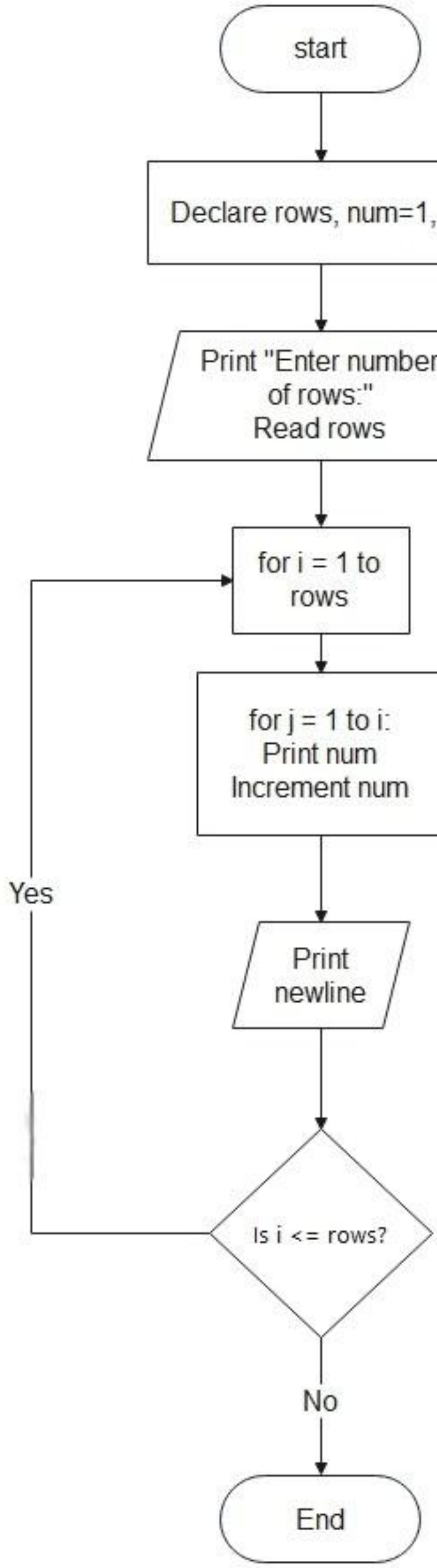
Code:

```c
#include <stdio.h>

int main() {
   int rows, num = 1, i, j;
   printf("Enter the number of rows: ");
   scanf("%d", &rows);

   for(i = 1; i <= rows; i++) {
      for(j = 1; j <= i; j++) {
         printf("%d ", num);
         num++;
      }
      printf("\n");
   }
   return 0;
}
```

Flowchart:



start
Declare rows, num=1, i, j
Print "Enter number of rows:"
Read rows
for i = 1 to rows
for j = 1 to i:
Print num
Increment num
Print newline
Is i <= rows?
Yes
No
End

4. **Write a c program to display diamond star pattern**

Algorithm:

1. Start
2. Input the number n (half the height of the diamond)
3. Upper half of the diamond:
    - Loop from i = 1 to n:
        - Print (n - i) spaces
        - Print (2 * i - 1) stars
4. Lower half of the diamond:
    - Loop from i = n-1 down to 1:
        - Print (n - i) spaces
        - Print (2 * i - 1) stars
5. End

Code:

```
#include <stdio.h>

int main() {
   int i, j, space, n;
   printf("Enter the number of rows (half of diamond height): ");
   scanf("%d", &n);

   // Upper half
   for(i = 1; i <= n; i++) {
      for(space = 1; space <= n - i; space++)
         printf(" ");
      for(j = 1; j <= 2 * i - 1; j++)
         printf("*");
      printf("\n");
   }

   // Lower half
   for(i = n-1; i >= 1; i--) {
      for(space = 1; space <= n - i; space++)
         printf(" ");
      for(j = 1; j <= 2 * i - 1; j++)
         printf("*");
      printf("\n");
   }
```

```
    return 0;
}
```

Flowchart: